COMPTIA SECURITY+ SY0-701 | DOMÍNIO 4 · OPERAÇÕES DE SEGURANÇA (28%)

Security+ 4.1 · Application Security

Mestre CyberSec · **Aula 4.1 — Application Security** (8:26) · Assistir Vídeo no YouTube ↗ · Treino Interativo: edusouzamarques.github.io/cybersec

Resumo Executivo da Aula: AppSec: input validation (NUNCA confie no input — sanitize tudo); secure cookie (HttpOnly, Secure, SameSite); code signing (assinatura do executável); sandboxing; SAST (static code analysis).

💡 Highlights que a prova CompTIA cobra

- Input validation: base de TUDO — sanitize/escape/whitelist
- HttpOnly cookie: JS não lê — mitiga stealing (XSS)

🎯 Dicas de Prova & Cenários Típicos da Banca

- 'JavaScript cannot access=this cookie'=HttpOnly

🧯 Além do Vídeo: Conceitos Oficiais, Siglas e Armadilhas (Seção 4.1 — Application Security)

Por que este aprofundamento é cobrado na prova?: O vídeo resume os tópicos práticos, mas o exame CompTIA SY0-701 cobra discernimento técnico rigoroso sobre as tecnologias e protocolos específicos desta seção:

• Validação de Entrada (N/A)

Processo de verificar e sanitizar todos os dados externos ou fornecidos pelo usuário antes do processamento para evitar injeção, XSS e entradas malformadas. É a base da codificação segura de aplicações.

⚠ ***Armadilha CompTIA:*** *CompTIA: validação de entrada é a base de TUDO; use whitelist/allowlist, sanitize e escape. Não confie em nenhum input.*

• Cookie HttpOnly

Atributo de cookie que impede o JavaScript do lado do cliente de acessar o cookie via document.cookie, reduzindo roubo de sessão por XSS.

⚠ ***Armadilha CompTIA:*** *Gatilho clássico: 'JavaScript cannot access this cookie' = HttpOnly. Não substitui Secure nem SameSite.*

• Atributo Secure de Cookie

Atributo de cookie que garante que o cookie só seja transmitido por conexões HTTPS/TLS criptografadas, protegendo-o de interceptação em texto claro.

⚠ ***Armadilha CompTIA:*** *CompTIA: Secure = somente HTTPS. Se a questão fala de confidencialidade no transporte do cookie, é Secure, não HttpOnly.*

• Atributo SameSite de Cookie

Atributo de cookie que controla se cookies são enviados em requisições entre sites, ajudando a mitigar CSRF e rastreamento cross-site.

⚠ ***Armadilha CompTIA:*** *Gatilho: proteção contra CSRF/cross-site request = SameSite. Valores comuns: Strict, Lax, None (None exige Secure).*

• Assinatura de Código (N/A)

Uso de assinatura digital em executáveis, scripts ou aplicações para verificar autenticidade do publicador e integridade, garantindo que o código não foi alterado.

⚠ ***Armadilha CompTIA:*** *CompTIA: assinatura do executável = code signing. Distingue de hashing simples: fornece autenticidade e integridade via certificado do desenvolvedor.*

• Sandbox / Isolamento de Execução (N/A)

Técnica de isolamento que executa código ou aplicações em ambiente restrito para limitar acesso a recursos do sistema e evitar danos de código malicioso ou não confiável.

⚠️ ***Armadilha CompTIA:*** *Gatilho: executar código não confiável com baixo impacto = sandbox. Não é antivírus; é contenção/isolamento.*

• Teste Estático de Segurança de Aplicações (SAST)

Método de teste que analisa código-fonte, bytecode ou binários sem executar a aplicação para encontrar falhas de segurança precocemente no SDLC.

⚠️ ***Armadilha CompTIA:*** *CompTIA: SAST = analisa código sem executar (white-box). Não confunda com DAST, que testa a aplicação em execução (black-box).*

• Scripting entre Sites (XSS)

Vulnerabilidade web em que o atacante injeta scripts maliciosos em sites confiáveis visualizados por outros usuários, frequentemente roubando cookies ou tokens de sessão.

⚠️ ***Armadilha CompTIA:*** *Gatilho: script malicioso executado no navegador da vítima = XSS. Mitigações: input validation, output encoding, HttpOnly cookies, CSP.*

✅ Perguntas de Treino Comentadas (10 Questões de Cenários Reais)

Cenários autênticos alinhados aos objetivos do exame. O gabarito com análise completa dos distratores encontra-se ao final.

QUESTÃO 1 · CENÁRIO DE EXAME

Uma aplicação web exibe o termo de busca do usuário na página de resultados inserindo-o diretamente na resposta HTML. Testes de segurança mostram que um atacante pode enviar um termo de busca contendo JavaScript que executa no navegador da vítima. Qual controle deve ser implementado para tratar essa vulnerabilidade em sua origem?

A) Adicionar o atributo HttpOnly ao cookie de sessão.

B) Realizar codificação de saída contextual e validação de entrada no servidor.

C) Configurar o cookie de sessão com SameSite=Strict.

D) Exigir assinatura de código para os binários da aplicação.

QUESTÃO 2 · CENÁRIO DE EXAME

O cookie de sessão de um varejista online é acessível por JavaScript, e o cookie é transmitido mesmo quando os usuários se conectam por HTTP não criptografado. Um testador de penetração alerta que uma única falha de cross-site scripting ou sniffing de rede pode levar ao sequestro de sessão. Qual conjunto de atributos de cookie o desenvolvedor deve aplicar ao cookie de sessão?

A) HttpOnly, Secure, SameSite=Strict

B) Domain, Path, Expires

C) SameSite=None, Secure=false

D) Max-Age, Priority=High

QUESTÃO 3 · CENÁRIO DE EXAME

Uma aplicação bancária usa um cookie de sessão que é enviado automaticamente em todas as requisições de qualquer site. Um analista de segurança demonstra que um site malicioso pode enviar um formulário oculto ao banco e disparar uma transferência autenticada de fundos. Qual alteração de atributo de cookie mitiga melhor esse risco de cross-site request forgery?

A) Adicionar o atributo Secure.

B) Adicionar o atributo HttpOnly.

C) Definir SameSite=Strict no cookie de sessão.

D) Aumentar o valor de Max-Age do cookie.

QUESTÃO 4 · CENÁRIO DE EXAME

Um fornecedor de software distribui um cliente desktop Windows por seu site público. Usuários começaram a receber avisos de que o publicador não pode ser verificado, e atacantes podem distribuir um instalador modificado. Qual controle o fornecedor deve implementar para permitir que os usuários verifiquem a origem e a integridade do software?

A) Executar SAST no código-fonte do cliente desktop.

B) Implantar o cliente em um contêiner sandbox.

C) Adicionar flags HttpOnly e SameSite ao cookie de sessão web.

D) Assinar o executável e o instalador com um certificado de assinatura de código confiável.

QUESTÃO 5 · CENÁRIO DE EXAME

Um analista de malware precisa abrir anexos de e-mail suspeitos e observar seu comportamento sem arriscar a estação de trabalho corporativa ou deixar a amostra contatar servidores de comando e controle. Qual abordagem é mais apropriada?

A) Assinar os anexos com código antes de abri-los.

B) Executar os anexos em um sandbox isolado com acesso de rede restrito.

C) Executar uma varredura SAST nos anexos de e-mail.

D) Definir os cookies do cliente de e-mail como HttpOnly.

QUESTÃO 6 · CENÁRIO DE EXAME

Uma equipe de desenvolvimento quer identificar padrões de codificação inseguros, como credenciais embutidas e consultas de banco de dados não sanitizadas, antes que a aplicação seja compilada e implantada. Qual atividade de segurança de aplicação deve ser integrada ao pipeline de CI/CD?

A) Teste estático de segurança de aplicação (SAST)

B) Teste dinâmico de segurança de aplicação (DAST)

C) Assinatura de código

D) Sandboxing

QUESTÃO 7 · CENÁRIO DE EXAME

Uma aplicação web permite que usuários enviem fotos de perfil. Um testador de penetração conseguiu enviar um arquivo chamado avatar.php contendo código do lado do servidor e depois acessá-lo por uma URL. Qual controle deve ser implementado para prevenir essa vulnerabilidade de upload de arquivos?

A) Adicionar o atributo Secure ao cookie de sessão.

B) Assinar os arquivos executáveis da aplicação web.

C) Validar tipo e extensão de arquivo, e armazenar arquivos enviados fora da raiz web com nomes aleatórios.

D) Definir o cookie de sessão como SameSite=Lax.

QUESTÃO 8 · CENÁRIO DE EXAME

Durante uma revisão de segurança, um desenvolvedor confirma que um cookie de sessão está marcado como Secure e SameSite=Lax, mas uma falha de cross-site scripting ainda permite que um atacante leia o valor do cookie por document.cookie. Qual atributo de cookie deve ser adicionado para impedir acesso JavaScript ao cookie de sessão?

A) Secure

B) SameSite=Strict

C) Domain

D) HttpOnly

QUESTÃO 9 · CENÁRIO DE EXAME

Uma organização quer garantir que apenas aplicações internas aprovadas e não modificadas possam ser executadas em endpoints de funcionários. A segurança de endpoint deve verificar se cada aplicação carrega uma assinatura digital válida do publicador de software antes de permitir a execução. Qual controle de segurança de aplicação isso descreve?

A) Verificação de assinatura de código com allowlisting de aplicações

B) Validação de entrada

C) Teste estático de segurança de aplicação (SAST)

D) Sandboxing

QUESTÃO 10 · CENÁRIO DE EXAME

Um engenheiro de segurança está revisando o código-fonte de um novo serviço de pagamento. O engenheiro quer encontrar possíveis falhas de injeção e uso inseguro de API sem executar a aplicação ou atacar um ambiente em execução. Qual técnica deve ser usada?

A) Teste dinâmico de segurança de aplicação (DAST)

B) Teste estático de segurança de aplicação (SAST)

C) Sandboxing

D) Configuração segura de cookie

GABARITO COMENTADO COM ANÁLISE DE DISTRATORES

Questão 1: Alternativa B — Realizar codificação de saída contextual e validação de entrada no servidor.
A vulnerabilidade é cross-site scripting causada por entrada de usuário não codificada. Codificação de saída contextual impede que o navegador trate dados do usuário como script executável, e validação de entrada no servidor reduz entradas maliciosas. HttpOnly, SameSite e assinatura de código protegem outras áreas, como roubo de cookie ou integridade de binário, mas não corrigem a causa raiz do XSS refletido.

Questão 2: Alternativa A — HttpOnly, Secure, SameSite=Strict
HttpOnly impede que JavaScript leia o cookie, Secure garante que ele seja enviado apenas por HTTPS, e SameSite=Strict limita o envio entre sites. Domain, Path, Expires, Max-Age e Priority não fornecem as mesmas proteções de confidencialidade e anti-CSRF.

Questão 3: Alternativa C — Definir SameSite=Strict no cookie de sessão.
SameSite=Strict impede que o navegador envie o cookie de sessão em requisições entre sites, o que bloqueia o ataque CSRF demonstrado. Secure protege a confidencialidade do cookie em trânsito, HttpOnly bloqueia acesso por JavaScript, e Max-Age afeta persistência, mas nenhum deles impede diretamente envios de formulário entre sites.

Questão 4: Alternativa D — Assinar o executável e o instalador com um certificado de assinatura de código confiável.
Assinatura de código usa uma assinatura digital para provar a identidade do publicador e confirmar que o binário não foi alterado. SAST encontra falhas no código-fonte, sandboxing limita o impacto em tempo de execução, e flags de cookie protegem sessões web, mas nenhum deles permite ao usuário verificar origem e integridade do instalador.

Questão 5: Alternativa B — Executar os anexos em um sandbox isolado com acesso de rede restrito.
Sandboxing executa código não confiável em um ambiente isolado e pode restringir acesso à rede, permitindo observação segura de comportamento. Assinatura de código verifica software confiável, mas não torna malware seguro; SAST analisa código-fonte; e HttpOnly é um controle de cookie web.

Questão 6: Alternativa A — Teste estático de segurança de aplicação (SAST)
SAST examina código-fonte ou binários sem executar a aplicação, sendo adequado para encontrar segredos embutidos e padrões de consulta inseguros no início do CI/CD. DAST testa uma aplicação em execução, assinatura de código verifica integridade, e sandboxing isola a execução.

Questão 7: Alternativa C — Validar tipo e extensão de arquivo, e armazenar arquivos enviados fora da raiz web com nomes aleatórios.
A aplicação deve validar uploads e impedir que arquivos executáveis controlados pelo usuário sejam servidos. Restringir tipo e extensão, armazenar arquivos fora da raiz web e randomizar nomes reduz a chance de execução de código. Atributos de cookie e assinatura de código não tratam o manuseio inseguro de upload de arquivos.

Questão 8: Alternativa D — HttpOnly
HttpOnly impede que scripts do lado do cliente acessem o cookie por document.cookie, reduzindo roubo de sessão via XSS. Secure protege a transmissão, SameSite limita requisições entre sites, e Domain define o escopo de envio do cookie, mas nenhum bloqueia acesso por JavaScript.

Questão 9: Alternativa A — Verificação de assinatura de código com allowlisting de aplicações
A verificação de assinatura de código checa a assinatura digital do publicador e a integridade, e a allowlisting pode permitir apenas aplicações assinadas e aprovadas de executar. Validação de entrada, SAST e sandboxing são controles valiosos, mas não impõem autenticidade do publicador no momento da execução no endpoint.

Questão 10: Alternativa B — Teste estático de segurança de aplicação (SAST)
SAST analisa código-fonte, bytecode ou binários sem executar a aplicação, portanto pode identificar falhas de injeção e uso inseguro de API antecipadamente. DAST testa uma aplicação em execução, sandboxing isola a execução, e configurações seguras de cookie tratam segurança de sessão, não falhas de código-fonte.